ASSURE DEFI®

THE VERIFICATION GOLD STANDARD

SECURITY ASSESSMENT REPORT

VERIFIED BY ASSURE DEFI®

INTEGRITY • TRUST • CREDIBILITY

NAME:	ZYPTO TOKEN
STATUS:	FAIL
DATE:	18/03/2026

Risk Analysis

Vulnerability summary

Classification	Description
High	Vulnerabilities that can lead to **loss or theft of funds**, **permanent locking of assets**, **unauthorized minting/burning**, **protocol insolvency**, or **full control of the contract or protocol**. These issues are typically **exploitable on-chain** and require immediate remediation.
Medium	Vulnerabilities that **weaken protocol security or trust assumptions** but do not directly enable immediate loss of funds under normal conditions. Exploitation may require **specific conditions, privileged roles, or external failures**.
Low	Issues that have **limited direct impact** on funds or protocol integrity but may cause **unexpected behavior**, **reduced resilience**, or **future exploitability** if combined with other weaknesses.
Informational	Findings that **do not represent security vulnerabilities**, but highlight **code quality, clarity, maintainability, or best-practice improvements** that may enhance long-term safety and auditability.

SCOPE

Target Code And Revision

Project	ZyptoToken
Codebase	https://bscscan.com/token/0xb838fb4edc798D0D8Ff3B4e3CAA9FFE318c620B7
Audit Methodology	Static, Manual

Detailed Technical Report

HIGH

1. Over-privileged CCIP mint authority enables arbitrary supply inflation and LP value extraction

Location:

ZyptoToken.sol - mint(address account, uint256 amount) and ZyptoToken.sol - setTokenPool(address _tokenPool)

Issue:

mint() allows either tokenPool or owner() to mint arbitrary amounts of tokens:

```
require(msg.sender == tokenPool || msg.sender == owner(), "Not authorized");
```

In a CCIP / bridge design, minting by the bridge pool is expected. The issue is that the owner also permanently retains direct mint authority, and can additionally replace tokenPool at any time through setTokenPool().

This creates an over-privileged trust model where:

the owner can mint unlimited supply outside the intended bridge flow

a compromised owner can assign a malicious tokenPool

newly minted tokens can be sold into the AMM pair to extract the paired

asset and collapse token price

Remediation:

Restrict mint() to the bridge pool only:

```
require(msg.sender == tokenPool, "Not authorized");
```

Harden setTokenPool() by:

allowing it only once, or

protecting it with a timelocked multisig / governance process

validating non-zero and contract addresses

If bridge upgrades are required, use a 2-step governed pool rotation instead of unrestricted owner replacement.

2. Over-privileged CCIP burn authority enables arbitrary user balance destruction

Location:

ZyptoToken.sol - burn(address account, uint256 amount)

Issue:

burn() allows either tokenPool or owner() to burn tokens from any arbitrary account:

```
require(msg.sender == tokenPool || msg.sender == owner(), "Not authorized");
_burn(account, amount);
```

For CCIP bridge flows, burning from a user address can be legitimate when performed only by the authorized bridge pool after validating a bridge-out operation.

The issue is that the contract also grants this power directly to owner(), which means the owner can destroy tokens from any wallet without approval, allowance, or bridge interaction.

This allows:

confiscation of user balances

censorship of holders

arbitrary destruction of treasury / LP / operational balances

irreversible loss of funds if the owner key is compromised

Remediation:

Restrict burn() to the bridge pool only:

```
require(msg.sender == tokenPool, "Not authorized");
```

If a self-burn mechanism is desired, implement a separate user-controlled burn(uint256 amount) function.

Do not allow the owner to arbitrarily burn balances from third-party accounts.

MEDIUM

1. Slippage-free auto-swaps allow sandwich extraction of fee inventory

Location:

ZyptoToken.sol - _swapTokensForETH(uint256 tokenAmount, address to)

Issue:

The contract swaps tokens for ETH with amountOutMin = 0:

```
_uniswapV2Router.swapExactTokensForETHSupportingFeeOnTransferTokens(
    tokenAmount, 0, path, to, block.timestamp
);
```

This means the contract accepts any output amount, regardless of price movement. Because these auto-swaps are predictable and triggered on sells, they can be sandwiched by MEV searchers:

- attacker front-runs the swap and pushes price against the token
- the contract executes at a poor rate
- attacker back-runs and captures the price dislocation

This does not usually drain the entire protocol in one transaction, but it can repeatedly extract value from the contract's accumulated fee inventory and worsen execution for liquidity/marketing flows.

Remediation:

Add slippage protection:

- quote expected output before swapping
- enforce a minimum acceptable amount out
- optionally cap execution to a TWAP- or oracle-derived tolerance band

Also consider splitting large swaps into bounded chunks to reduce MEV attractiveness.

2. Full-balance swap trigger causes excessive price impact and predictable griefing / MEV surface

Location:

ZyptoToken.sol - _trade(address from, address to, uint256 value)

ZyptoToken.sol - _swapAndLiquify(uint256 tokenAmount)

Issue:

On sells, once the threshold is reached, the contract swaps the entire token balance held by the contract:

```
uint256 contractBalance = balanceOf(address(this));
if (contractBalance >= _swapTokensAtAmount) {
    _swapAndLiquify(contractBalance);
}
```

This design can create a large forced sell event on the next triggering transaction. As a result:

- the triggering seller may suffer unexpectedly high slippage
- MEV bots can anticipate and exploit the large contract swap
- an attacker can grief execution by pushing more fee-bearing transfers into the contract balance before the trigger
- price impact becomes significantly worse than necessary

The issue is primarily economic and execution-related, rather than a direct arbitrary theft primitive.

Remediation:

Cap the swap size per execution. For example:

```
uint256 swapAmount = contractBalance > maxSwapAmount ? maxSwapAmount :
contractBalance;
```

Use a bounded maxSwapAmount and perform incremental processing across transactions instead of swapping the full balance at once.

LOW

No low severity issues were found.

INFORMATIONAL

No informational severity issues were found.

Testing coverage

During the testing phase, custom use cases were written to cover all the logic of contracts. *Check "Annexes" to see the testing code.*

```
ZyptoToken
  deployment and configuration
    ✓ sets constructor state correctly (1702ms)
    ✓ sets default fee structure and exclusions (67ms)
    ✓ gives the router max allowance from the token contract (53ms)
    ✓ emits TradingOpened when trading is enabled (46ms)
    ✓ allows the owner to update swap threshold (48ms)
    ✓ allows the owner to toggle swapAndLiquify (56ms)
    ✓ lets the owner set a tokenPool and auto-excludes it from fees (48ms)
    ✓ lets the owner update the marketing wallet and auto-excludes it (42ms)
    ✓ keeps the previous marketing wallet excluded after changing it (39ms)
    ✓ lets the owner exclude and re-include an address from fees (47ms)
  access control
    ✓ only owner can open trading (48ms)
    ✓ only owner can set swap amount
    ✓ only owner can toggle swapAndLiquify
    ✓ only owner can set fees (41ms)
    ✓ only owner can set the token pool
    ✓ only owner can set marketing wallet and fee exclusions
    ✓ only owner can configure AMM pairs (41ms)
    ✓ only owner can rescue tokens and ETH (40ms)
    ✓ allows only owner or tokenPool to mint and burn (50ms)
  trading controls and fee behavior
    ✓ blocks sells before openTrade while allowing excluded transfers (47ms)
    ✓ blocks buys before openTrade (42ms)
    ✓ allows normal wallet-to-wallet transfers before openTrade (46ms)
    ✓ applies buy fees correctly after trading opens (47ms)
    ✓ applies sell fees correctly after trading opens (40ms)
    ✓ does not charge fees for an excluded seller (40ms)
    ✓ does not charge fees for an excluded buyer (41ms)
    ✓ allows tokenPool transfers with trading closed (39ms)
    ✓ tracks pending marketing value across multiple buys (41ms)
    ✓ tracks pending marketing value across multiple sells (50ms)
    ✓ lets the owner update buy fees within the cap (44ms)
    ✓ lets the owner update sell fees within the cap (45ms)
    ✓ enforces the max fee cap (45ms)
  AMM pair handling
    ✓ cannot remove the primary pair (40ms)
    ✓ can add a secondary AMM pair (47ms)
    ✓ can remove a secondary AMM pair (54ms)
    ✓ applies buy fees when buying from a secondary AMM pair (48ms)
    ✓ applies sell fees when selling into a secondary AMM pair (56ms)
  swap and liquify flow
    ✓ triggers swap before current sell tax is added (78ms)
    ✓ does not trigger swap when disabled (59ms)
    ✓ does not trigger swap when contract balance is below threshold (54ms)
    ✓ can perform more than one swap cycle across multiple sells (63ms)
    ✓ emits SwapAndLiquify with the liquidity half and ETH received (62ms)
    ✓ reduces pendingMarketingValue by the swapped pending amount before adding current sell marketing (48ms)
  rescue functions
    ✓ rescues ERC20 balances to the owner (68ms)
    ✓ rescues ETH to the owner (53ms)
    ✓ can rescue the token's own ERC20 balance to the owner (42ms)
  scenario and regression checks
    ✓ keeps total supply unchanged during taxed buy and sell cycles (61ms)
    ✓ supports sequential user trades without breaking accounting (72ms)
    ✓ handles zero pending marketing path when swap is triggered from manually preloaded balance (63ms)


49 passing (4s)
```

Annexes

Testing code:

```
const { expect } = require("chai");
const { ethers } = require("hardhat");

const E18 = 10n ** 18n;
const BUY_LIQ = 200n;
const BUY_MKT = 200n;
const SELL_LIQ = 200n;
const SELL_MKT = 200n;
const TOTAL_FEE = 400n;

function feeOf(amount, bps) {
  return (amount * bps) / 10000n;
}

async function deployFixture() {
  const [deployer, owner, marketing, user1, user2, tokenPool, outsider, treasury] = await
ethers.getSigners();

  const MockERC20 = await ethers.getContractFactory("MockERC20");
  const weth = await MockERC20.deploy("Wrapped Ether", "WETH");
  await weth.waitForDeployment();

  const MockRouter = await ethers.getContractFactory("MockRouter");
  const router = await MockRouter.deploy(await weth.getAddress(), {
    value: ethers.parseEther("100")
  });
  await router.waitForDeployment();

  const ZyptoToken = await ethers.getContractFactory("ZyptoToken");
  const token = await ZyptoToken.deploy(
    owner.address,
    await router.getAddress(),
```

```
    marketing.address
  );
  await token.waitForDeployment();

  const pair = await token._uniswapV2Pair();
  const pairSigner = await ethers.getImpersonatedSigner(pair);
  await ethers.provider.send("hardhat_setBalance", [pair, "0x1000000000000000000"]);

  async function buyTo(to, amount) {
    await token.connect(owner).mint(pair, amount);
    await token.connect(pairSigner).transfer(to.address ?? to, amount);
  }

  async function sellFrom(signer, amount) {
    await token.connect(signer).transfer(pair, amount);
  }

  return {
    deployer,
    owner,
    marketing,
    user1,
    user2,
    tokenPool,
    outsider,
    treasury,
    weth,
    router,
    token,
    pair,
    pairSigner,
    buyTo,
    sellFrom,
  };
}

describe("ZyptoToken", function () {
  describe("deployment and configuration", function () {
    it("sets constructor state correctly", async function () {
      const { owner, marketing, router, token, pair, weth } = await deployFixture();

      expect(await token.owner()).to.equal(owner.address);
      expect(await token.name()).to.equal("Zypto Token");
      expect(await token.symbol()).to.equal("ZYPTO");
      expect(await token.decimals()).to.equal(18);
```

```
      expect(await token._marketingWallet()).to.equal(marketing.address);
      expect(await token._uniswapV2Router()).to.equal(await router.getAddress());
      expect(await token._WETH()).to.equal(await weth.getAddress());
      expect(await token._uniswapV2Pair()).to.equal(pair);
      expect(await token._isAutomatedMarketMakerPair(pair)).to.equal(true);
      expect(await token._isOpenForTrading()).to.equal(false);
      expect(await token._swapAndLiquifyEnabled()).to.equal(true);
      expect(await token._swapTokensAtAmount()).to.equal(100_000n * E18);
      expect(await token.totalSupply()).to.equal(0n);
    });

    it("sets default fee structure and exclusions", async function () {
      const { owner, marketing, token } = await deployFixture();

      expect(await token._buyFeeLiquidity()).to.equal(BUY_LIQ);
      expect(await token._buyFeeMarketing()).to.equal(BUY_MKT);
      expect(await token._buyFeeTotal()).to.equal(TOTAL_FEE);
      expect(await token._sellFeeLiquidity()).to.equal(SELL_LIQ);
      expect(await token._sellFeeMarketing()).to.equal(SELL_MKT);
      expect(await token._sellFeeTotal()).to.equal(TOTAL_FEE);
      expect(await token._pendingMarketingValue()).to.equal(0n);
      expect(await token._isExcludedFromFees(owner.address)).to.equal(true);
      expect(await token._isExcludedFromFees(marketing.address)).to.equal(true);
      expect(await token._isExcludedFromFees(await token.getAddress())).to.equal(true);
    });

    it("gives the router max allowance from the token contract", async function () {
      const { router, token } = await deployFixture();
      expect(await token.allowance(await token.getAddress(), await
router.getAddress())).to.equal((2n ** 256n) - 1n);
    });

    it("emits TradingOpened when trading is enabled", async function () {
      const { owner, token } = await deployFixture();
      await expect(token.connect(owner).openTrade()).to.emit(token, "TradingOpened");
      expect(await token._isOpenForTrading()).to.equal(true);
    });

    it("allows the owner to update swap threshold", async function () {
      const { owner, token } = await deployFixture();
      await expect(token.connect(owner).setSwapAmount(123n * E18))
        .to.emit(token, "SetSwapTokensAtAmount")
        .withArgs(123n * E18);
      expect(await token._swapTokensAtAmount()).to.equal(123n * E18);
    });
```

```
  it("allows the owner to toggle swapAndLiquify", async function () {
    const { owner, token } = await deployFixture();
    await expect(token.connect(owner).setSwapAndLiquifyEnabled(false))
      .to.emit(token, "SetSwapAndLiquifyEnabled")
      .withArgs(false);
    expect(await token._swapAndLiquifyEnabled()).to.equal(false);
  });

  it("lets the owner set a tokenPool and auto-excludes it from fees", async function () {
    const { owner, tokenPool, token } = await deployFixture();
    await expect(token.connect(owner).setTokenPool(tokenPool.address))
      .to.emit(token, "SetTokenPool")
      .withArgs(tokenPool.address);
    expect(await token.tokenPool()).to.equal(tokenPool.address);
    expect(await token._isExcludedFromFees(tokenPool.address)).to.equal(true);
  });

  it("lets the owner update the marketing wallet and auto-excludes it", async function ()
{
    const { owner, user2, token } = await deployFixture();
    await expect(token.connect(owner).setMarketingWallet(user2.address))
      .to.emit(token, "SetMarketingWallet")
      .withArgs(user2.address);
    expect(await token._marketingWallet()).to.equal(user2.address);
    expect(await token._isExcludedFromFees(user2.address)).to.equal(true);
  });

  it("keeps the previous marketing wallet excluded after changing it", async function ()
{
    const { owner, marketing, user2, token } = await deployFixture();
    await token.connect(owner).setMarketingWallet(user2.address);
    expect(await token._isExcludedFromFees(marketing.address)).to.equal(true);
    expect(await token._isExcludedFromFees(user2.address)).to.equal(true);
  });

  it("lets the owner exclude and re-include an address from fees", async function () {
    const { owner, user1, token } = await deployFixture();
    await expect(token.connect(owner).excludeFromFees(user1.address, true))
      .to.emit(token, "ExcludeFromFees")
      .withArgs(user1.address, true);
    expect(await token._isExcludedFromFees(user1.address)).to.equal(true);

    await expect(token.connect(owner).excludeFromFees(user1.address, false))
      .to.emit(token, "ExcludeFromFees")
```

```
        .withArgs(user1.address, false);
      expect(await token._isExcludedFromFees(user1.address)).to.equal(false);
    });
  });

  describe("access control", function () {
    it("only owner can open trading", async function () {
      const { outsider, token } = await deployFixture();
      await expect(token.connect(outsider).openTrade()).to.be.reverted;
    });

    it("only owner can set swap amount", async function () {
      const { outsider, token } = await deployFixture();
      await expect(token.connect(outsider).setSwapAmount(1n)).to.be.reverted;
    });

    it("only owner can toggle swapAndLiquify", async function () {
      const { outsider, token } = await deployFixture();
      await expect(token.connect(outsider).setSwapAndLiquifyEnabled(false)).to.be.reverted;
    });

    it("only owner can set fees", async function () {
      const { outsider, token } = await deployFixture();
      await expect(token.connect(outsider).setBuyFee(100, 100)).to.be.reverted;
      await expect(token.connect(outsider).setSellFee(100, 100)).to.be.reverted;
    });

    it("only owner can set the token pool", async function () {
      const { outsider, tokenPool, token } = await deployFixture();
      await expect(token.connect(outsider).setTokenPool(tokenPool.address)).to.be.reverted;
    });

    it("only owner can set marketing wallet and fee exclusions", async function () {
      const { outsider, user1, token } = await deployFixture();
      await
expect(token.connect(outsider).setMarketingWallet(user1.address)).to.be.reverted;
      await expect(token.connect(outsider).excludeFromFees(user1.address,
true)).to.be.reverted;
    });

    it("only owner can configure AMM pairs", async function () {
      const { outsider, user2, token } = await deployFixture();
      await expect(token.connect(outsider).setAutomatedMarketMakerPair(user2.address,
true)).to.be.reverted;
    });
```

```
    it("only owner can rescue tokens and ETH", async function () {
      const { outsider, token } = await deployFixture();
      await expect(token.connect(outsider).rescueETH()).to.be.reverted;
      await expect(token.connect(outsider).rescueERC20(await
token.getAddress())).to.be.reverted;
    });

    it("allows only owner or tokenPool to mint and burn", async function () {
      const { owner, user1, tokenPool, outsider, token } = await deployFixture();

      await expect(token.connect(outsider).mint(user1.address, 1n)).to.be.revertedWith("Not
authorized");
      await expect(token.connect(outsider).burn(user1.address, 1n)).to.be.revertedWith("Not
authorized");

      await token.connect(owner).mint(user1.address, 10n * E18);
      expect(await token.balanceOf(user1.address)).to.equal(10n * E18);

      await token.connect(owner).setTokenPool(tokenPool.address);
      await token.connect(tokenPool).mint(user1.address, 5n * E18);
      expect(await token.balanceOf(user1.address)).to.equal(15n * E18);

      await token.connect(tokenPool).burn(user1.address, 3n * E18);
      expect(await token.balanceOf(user1.address)).to.equal(12n * E18);
    });
  });

  describe("trading controls and fee behavior", function () {
    it("blocks sells before openTrade while allowing excluded transfers", async function ()
{
      const { owner, user1, token, pair } = await deployFixture();

      await token.connect(owner).mint(user1.address, 100n * E18);
      await token.connect(owner).mint(owner.address, 100n * E18);

      await expect(token.connect(user1).transfer(pair, 1n * E18)).to.be.revertedWith("Not
open");
      await expect(token.connect(owner).transfer(user1.address, 1n *
E18)).to.not.be.reverted;
    });

    it("blocks buys before openTrade", async function () {
      const { owner, user1, token, pair, pairSigner } = await deployFixture();
      await token.connect(owner).mint(pair, 10n * E18);
```

```
      await expect(token.connect(pairSigner).transfer(user1.address, 1n *
E18)).to.be.revertedWith("Not open");
    });

    it("allows normal wallet-to-wallet transfers before openTrade", async function () {
      const { owner, user1, user2, token } = await deployFixture();
      await token.connect(owner).mint(user1.address, 7n * E18);
      await expect(token.connect(user1).transfer(user2.address, 2n *
E18)).to.not.be.reverted;
      expect(await token.balanceOf(user2.address)).to.equal(2n * E18);
    });

    it("applies buy fees correctly after trading opens", async function () {
      const { owner, user1, token, buyTo } = await deployFixture();
      await token.connect(owner).openTrade();

      await buyTo(user1, 100n * E18);

      expect(await token.balanceOf(user1.address)).to.equal(96n * E18);
      expect(await token.balanceOf(await token.getAddress())).to.equal(4n * E18);
      expect(await token._pendingMarketingValue()).to.equal(2n * E18);
    });

    it("applies sell fees correctly after trading opens", async function () {
      const { owner, user1, token, pair, sellFrom } = await deployFixture();
      await token.connect(owner).mint(user1.address, 100n * E18);
      await token.connect(owner).openTrade();

      await sellFrom(user1, 100n * E18);

      expect(await token.balanceOf(pair)).to.equal(96n * E18);
      expect(await token.balanceOf(await token.getAddress())).to.equal(4n * E18);
      expect(await token._pendingMarketingValue()).to.equal(2n * E18);
    });

    it("does not charge fees for an excluded seller", async function () {
      const { owner, user1, token, pair } = await deployFixture();
      await token.connect(owner).mint(user1.address, 100n * E18);
      await token.connect(owner).excludeFromFees(user1.address, true);
      await token.connect(owner).openTrade();

      await token.connect(user1).transfer(pair, 100n * E18);

      expect(await token.balanceOf(pair)).to.equal(100n * E18);
      expect(await token.balanceOf(await token.getAddress())).to.equal(0n);
```

```
      expect(await token._pendingMarketingValue()).to.equal(0n);
    });

    it("does not charge fees for an excluded buyer", async function () {
      const { owner, user1, token, pair, pairSigner } = await deployFixture();
      await token.connect(owner).openTrade();
      await token.connect(owner).excludeFromFees(user1.address, true);
      await token.connect(owner).mint(pair, 100n * E18);

      await token.connect(pairSigner).transfer(user1.address, 100n * E18);

      expect(await token.balanceOf(user1.address)).to.equal(100n * E18);
      expect(await token.balanceOf(await token.getAddress())).to.equal(0n);
      expect(await token._pendingMarketingValue()).to.equal(0n);
    });

    it("allows tokenPool transfers with trading closed", async function () {
      const { owner, tokenPool, user1, token } = await deployFixture();
      await token.connect(owner).setTokenPool(tokenPool.address);
      await token.connect(owner).mint(tokenPool.address, 50n * E18);
      await expect(token.connect(tokenPool).transfer(user1.address, 10n *
E18)).to.not.be.reverted;
      expect(await token.balanceOf(user1.address)).to.equal(10n * E18);
    });

    it("tracks pending marketing value across multiple buys", async function () {
      const { owner, user1, user2, token, buyTo } = await deployFixture();
      await token.connect(owner).openTrade();

      await buyTo(user1, 100n * E18);
      await buyTo(user2, 50n * E18);

      expect(await token._pendingMarketingValue()).to.equal(3n * E18);
      expect(await token.balanceOf(await token.getAddress())).to.equal(6n * E18);
    });

    it("tracks pending marketing value across multiple sells", async function () {
      const { owner, user1, user2, token, sellFrom } = await deployFixture();
      await token.connect(owner).mint(user1.address, 100n * E18);
      await token.connect(owner).mint(user2.address, 50n * E18);
      await token.connect(owner).openTrade();

      await sellFrom(user1, 100n * E18);
      await sellFrom(user2, 50n * E18);
```

```
      expect(await token._pendingMarketingValue()).to.equal(3n * E18);
      expect(await token.balanceOf(await token.getAddress())).to.equal(6n * E18);
    });

    it("lets the owner update buy fees within the cap", async function () {
      const { owner, token } = await deployFixture();
      await expect(token.connect(owner).setBuyFee(125, 175))
        .to.emit(token, "SetBuyFees")
        .withArgs(125, 175);
      expect(await token._buyFeeLiquidity()).to.equal(125n);
      expect(await token._buyFeeMarketing()).to.equal(175n);
      expect(await token._buyFeeTotal()).to.equal(300n);
    });

    it("lets the owner update sell fees within the cap", async function () {
      const { owner, token } = await deployFixture();
      await expect(token.connect(owner).setSellFee(175, 125))
        .to.emit(token, "SetSellFees")
        .withArgs(175, 125);
      expect(await token._sellFeeLiquidity()).to.equal(175n);
      expect(await token._sellFeeMarketing()).to.equal(125n);
      expect(await token._sellFeeTotal()).to.equal(300n);
    });

    it("enforces the max fee cap", async function () {
      const { owner, token } = await deployFixture();
      await expect(token.connect(owner).setBuyFee(401, 0)).to.be.revertedWith("Fee too
high");
      await expect(token.connect(owner).setSellFee(250, 151)).to.be.revertedWith("Fee too
high");
    });
  });

  describe("AMM pair handling", function () {
    it("cannot remove the primary pair", async function () {
      const { owner, token, pair } = await deployFixture();
      await expect(token.connect(owner).setAutomatedMarketMakerPair(pair,
false)).to.be.revertedWith("Cannot remove primary pair");
    });

    it("can add a secondary AMM pair", async function () {
      const { owner, user2, token } = await deployFixture();
      await expect(token.connect(owner).setAutomatedMarketMakerPair(user2.address, true))
        .to.emit(token, "SetAutomatedMarketMakerPair")
        .withArgs(user2.address, true);
```

```
      expect(await token._isAutomatedMarketMakerPair(user2.address)).to.equal(true);
    });

    it("can remove a secondary AMM pair", async function () {
      const { owner, user2, token } = await deployFixture();
      await token.connect(owner).setAutomatedMarketMakerPair(user2.address, true);
      await expect(token.connect(owner).setAutomatedMarketMakerPair(user2.address, false))
        .to.emit(token, "SetAutomatedMarketMakerPair")
        .withArgs(user2.address, false);
      expect(await token._isAutomatedMarketMakerPair(user2.address)).to.equal(false);
    });

    it("applies buy fees when buying from a secondary AMM pair", async function () {
      const { owner, user1, user2, token } = await deployFixture();
      await token.connect(owner).setAutomatedMarketMakerPair(user2.address, true);
      await token.connect(owner).openTrade();
      await token.connect(owner).mint(user2.address, 100n * E18);

      await token.connect(user2).transfer(user1.address, 100n * E18);

      expect(await token.balanceOf(user1.address)).to.equal(96n * E18);
      expect(await token.balanceOf(await token.getAddress())).to.equal(4n * E18);
      expect(await token._pendingMarketingValue()).to.equal(2n * E18);
    });

    it("applies sell fees when selling into a secondary AMM pair", async function () {
      const { owner, user1, user2, token } = await deployFixture();
      await token.connect(owner).setAutomatedMarketMakerPair(user2.address, true);
      await token.connect(owner).openTrade();
      await token.connect(owner).mint(user1.address, 100n * E18);

      await token.connect(user1).transfer(user2.address, 100n * E18);

      expect(await token.balanceOf(user2.address)).to.equal(96n * E18);
      expect(await token.balanceOf(await token.getAddress())).to.equal(4n * E18);
      expect(await token._pendingMarketingValue()).to.equal(2n * E18);
    });
  });

  describe("swap and liquify flow", function () {
    it("triggers swap before current sell tax is added", async function () {
      const { owner, marketing, user1, router, token, pair, buyTo } = await
deployFixture();

      await token.connect(owner).openTrade();
```

```
      await token.connect(owner).setSwapAmount(4n * E18);

      await buyTo(user1, 100n * E18);
      expect(await token.balanceOf(await token.getAddress())).to.equal(4n * E18);
      expect(await token._pendingMarketingValue()).to.equal(2n * E18);

      const marketingBalanceBefore = await ethers.provider.getBalance(marketing.address);
      const routerTokenBalanceBefore = await token.balanceOf(await router.getAddress());

      const sellAmount = await token.balanceOf(user1.address);
      const expectedSellTax = feeOf(sellAmount, TOTAL_FEE);
      const expectedMarketingPortion = feeOf(sellAmount, SELL_MKT);

      await token.connect(user1).transfer(pair, sellAmount);

      const marketingBalanceAfter = await ethers.provider.getBalance(marketing.address);
      const routerTokenBalanceAfter = await token.balanceOf(await router.getAddress());

      expect(marketingBalanceAfter - marketingBalanceBefore).to.equal(2n * E18);
      expect(routerTokenBalanceAfter - routerTokenBalanceBefore).to.equal(4n * E18);
      expect(await token.balanceOf(await token.getAddress())).to.equal(expectedSellTax);
      expect(await token._pendingMarketingValue()).to.equal(expectedMarketingPortion);
    });

    it("does not trigger swap when disabled", async function () {
      const { owner, marketing, user1, token, buyTo } = await deployFixture();

      await token.connect(owner).openTrade();
      await token.connect(owner).setSwapAmount(4n * E18);
      await token.connect(owner).setSwapAndLiquifyEnabled(false);

      await buyTo(user1, 100n * E18);
      const marketingBalanceBefore = await ethers.provider.getBalance(marketing.address);
      const initialContractBalance = await token.balanceOf(await token.getAddress());
      const sellAmount = await token.balanceOf(user1.address);
      const expectedSellTax = feeOf(sellAmount, TOTAL_FEE);
      const expectedMarketingPortion = 2n * E18 + feeOf(sellAmount, SELL_MKT);

      await token.connect(user1).transfer(await token._uniswapV2Pair(), sellAmount);

      expect(await
ethers.provider.getBalance(marketing.address)).to.equal(marketingBalanceBefore);
      expect(await token.balanceOf(await
token.getAddress())).to.equal(initialContractBalance + expectedSellTax);
      expect(await token._pendingMarketingValue()).to.equal(expectedMarketingPortion);
```

```
    });

    it("does not trigger swap when contract balance is below threshold", async function ()
{
      const { owner, marketing, user1, token } = await deployFixture();
      await token.connect(owner).openTrade();
      await token.connect(owner).setSwapAmount(10n * E18);
      await token.connect(owner).mint(await token.getAddress(), 3n * E18);
      await token.connect(owner).mint(user1.address, 100n * E18);

      const marketingBefore = await ethers.provider.getBalance(marketing.address);
      await token.connect(user1).transfer(await token._uniswapV2Pair(), 100n * E18);

      expect(await
ethers.provider.getBalance(marketing.address)).to.equal(marketingBefore);
      expect(await token.balanceOf(await token.getAddress())).to.equal(7n * E18);
      expect(await token._pendingMarketingValue()).to.equal(2n * E18);
    });

    it("can perform more than one swap cycle across multiple sells", async function () {
      const { owner, marketing, user1, user2, token, buyTo } = await deployFixture();
      await token.connect(owner).openTrade();
      await token.connect(owner).setSwapAmount(4n * E18);

      await buyTo(user1, 100n * E18);
      const marketingBeforeCycle1 = await ethers.provider.getBalance(marketing.address);
      const user1Bal = await token.balanceOf(user1.address);
      await token.connect(user1).transfer(await token._uniswapV2Pair(), user1Bal);
      const marketingAfterCycle1 = await ethers.provider.getBalance(marketing.address);
      expect(marketingAfterCycle1 - marketingBeforeCycle1).to.equal(2n * E18);

      await buyTo(user2, 100n * E18);
      const marketingBeforeCycle2 = await ethers.provider.getBalance(marketing.address);
      const user2Bal = await token.balanceOf(user2.address);
      await token.connect(user2).transfer(await token._uniswapV2Pair(), user2Bal);
      const marketingAfterCycle2 = await ethers.provider.getBalance(marketing.address);
      expect(marketingAfterCycle2 - marketingBeforeCycle2).to.equal(3920000000000000000n);
    });

    it("emits SwapAndLiquify with the liquidity half and ETH received", async function () {
      const { owner, user1, token, buyTo } = await deployFixture();
      await token.connect(owner).openTrade();
      await token.connect(owner).setSwapAmount(4n * E18);
      await buyTo(user1, 100n * E18);
```

```
      const userBalance = await token.balanceOf(user1.address);
      await expect(token.connect(user1).transfer(await token._uniswapV2Pair(),
userBalance))
        .to.emit(token, "SwapAndLiquify")
        .withArgs(1n * E18, 1n * E18);
    });

    it("reduces pendingMarketingValue by the swapped pending amount before adding current
sell marketing", async function () {
      const { owner, user1, token, buyTo } = await deployFixture();
      await token.connect(owner).openTrade();
      await token.connect(owner).setSwapAmount(6n * E18);

      await buyTo(user1, 150n * E18);
      expect(await token._pendingMarketingValue()).to.equal(3n * E18);
      const sellAmount = await token.balanceOf(user1.address);
      await token.connect(user1).transfer(await token._uniswapV2Pair(), sellAmount);

      expect(await token._pendingMarketingValue()).to.equal(feeOf(sellAmount, SELL_MKT));
    });
  });

  describe("rescue functions", function () {
    it("rescues ERC20 balances to the owner", async function () {
      const { owner, token } = await deployFixture();
      const MockERC20 = await ethers.getContractFactory("MockERC20");
      const rescueToken = await MockERC20.deploy("Rescue", "RSC");
      await rescueToken.waitForDeployment();

      await rescueToken.mint(await token.getAddress(), 50n * E18);
      await expect(token.connect(owner).rescueERC20(await rescueToken.getAddress()))
        .to.emit(token, "EmergencyRescueERC20")
        .withArgs(await rescueToken.getAddress(), 50n * E18);
      expect(await rescueToken.balanceOf(owner.address)).to.equal(50n * E18);
    });

    it("rescues ETH to the owner", async function () {
      const { owner, token } = await deployFixture();
      await owner.sendTransaction({ to: await token.getAddress(), value:
ethers.parseEther("1") });

      const ownerEthBefore = await ethers.provider.getBalance(owner.address);
      const tx = await token.connect(owner).rescueETH();
      const receipt = await tx.wait();
      const gasPrice = receipt.gasPrice ?? tx.gasPrice ?? 0n;
```

```
      const gasCost = receipt.gasUsed * gasPrice;
      const ownerEthAfter = await ethers.provider.getBalance(owner.address);

      expect(ownerEthAfter + gasCost).to.be.greaterThanOrEqual(ownerEthBefore +
ethers.parseEther("1"));
      expect(await ethers.provider.getBalance(await token.getAddress())).to.equal(0n);
    });

    it("can rescue the token's own ERC20 balance to the owner", async function () {
      const { owner, token } = await deployFixture();
      await token.connect(owner).mint(await token.getAddress(), 11n * E18);
      await token.connect(owner).rescueERC20(await token.getAddress());
      expect(await token.balanceOf(owner.address)).to.equal(11n * E18);
      expect(await token.balanceOf(await token.getAddress())).to.equal(0n);
    });
  });

  describe("scenario and regression checks", function () {
    it("keeps total supply unchanged during taxed buy and sell cycles", async function () {
      const { owner, user1, token, pair, pairSigner } = await deployFixture();
      await token.connect(owner).openTrade();
      await token.connect(owner).mint(pair, 100n * E18);
      const supplyBefore = await token.totalSupply();
      await token.connect(pairSigner).transfer(user1.address, 100n * E18);
      const sellAmount = await token.balanceOf(user1.address);
      await token.connect(user1).transfer(pair, sellAmount);
      expect(await token.totalSupply()).to.equal(supplyBefore);
    });

    it("supports sequential user trades without breaking accounting", async function () {
      const { owner, user1, user2, token, buyTo } = await deployFixture();
      await token.connect(owner).openTrade();
      await buyTo(user1, 100n * E18);
      await buyTo(user2, 200n * E18);

      const contractBalAfterBuys = await token.balanceOf(await token.getAddress());
      expect(contractBalAfterBuys).to.equal(12n * E18);
      expect(await token._pendingMarketingValue()).to.equal(6n * E18);

      await token.connect(user1).transfer(await token._uniswapV2Pair(), await
token.balanceOf(user1.address));
      await token.connect(user2).transfer(await token._uniswapV2Pair(), await
token.balanceOf(user2.address));

      expect(await token.balanceOf(await token.getAddress())).to.be.greaterThanOrEqual(0n);
```

```
      expect(await token._pendingMarketingValue()).to.be.greaterThanOrEqual(0n);
    });

    it("handles zero pending marketing path when swap is triggered from manually preloaded
balance", async function () {
      const { owner, user1, marketing, token } = await deployFixture();
      await token.connect(owner).openTrade();
      await token.connect(owner).setSwapAmount(4n * E18);
      await token.connect(owner).mint(await token.getAddress(), 4n * E18);
      await token.connect(owner).mint(user1.address, 100n * E18);

      const marketingBefore = await ethers.provider.getBalance(marketing.address);
      await token.connect(user1).transfer(await token._uniswapV2Pair(), 100n * E18);

      expect(await
ethers.provider.getBalance(marketing.address)).to.equal(marketingBefore);
      expect(await token.balanceOf(await token.getAddress())).to.equal(4n * E18);
      expect(await token._pendingMarketingValue()).to.equal(2n * E18);
    });
  });
});
```

Technical Findings Summary

Findings

Vulnerability Level	Total	Pending	Not Apply	Acknowledged	Partially Fixed	Fixed
High	2					
Medium	2					
Low						
Informational						

Assessment Results

Score Results

Review	Score
Global Score	**70/100**
Assure KYC	Not completed
Audit Score	70/100

The Following Score System Has been Added to this page to help understand the value of the audit, the maximum score is 100, however to attain that value the project must pass and provide all the data needed for the assessment. Our Passing Score has been changed to 84 Points for a higher standard, if a project does not attain 85% is an automatic failure. Read our notes and final assessment below. The Global Score is a combination of the evaluations obtained between having or not having KYC and the type of contract audited together with its manual audit.

Audit FAIL

Following our comprehensive security audit of the smart contract for the **ZYPTOTOKEN** project, the project did not fulfill the necessary criteria required to pass the security audit.

Disclaimer

Assure Defi has conducted an independent security assessment to verify the integrity of and highlight any vulnerabilities or errors, intentional or unintentional, that may be present in the reviewed code for the scope of this assessment. This report does not constitute agreement, acceptance, or advocating for the Project, and users relying on this report should not consider this as having any merit for financial Token in any shape, form, or nature. The contracts audited do not account for any economic developments that the Project in question may pursue, and the veracity of the findings thus presented in this report relate solely to the proficiency, competence, aptitude, and discretion of our independent auditors, who make no guarantees nor assurance that the contracts are entirely free of exploits, bugs, vulnerabilities or deprecation of technologies. All information provided in this report does not constitute financial or investment in Token, nor should it be used to signal that any person reading this report should invest their funds without sufficient individual due diligence, regardless of the findings presented. Information is provided ‘as is, and Assure Defi is under no covenant to audit completeness, accuracy, or solidity of the contracts. In no event will Assure Defi or its partners, employees, agents, or parties related to the provision of this audit report be liable to any parties for, or lack thereof, decisions or actions with regards to the information provided in this audit report. The assessment of Tokens provided by Assure Defi are subject to dependencies and are under continuing development. You agree that your access or use, including but not limited to any Tokens, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies with high levels of technical risk and uncertainty. The assessment reports could include false positives, negatives, and unpredictable results. The Token may access, and depend upon, multiple layers of third parties.